Heart Disease Risk Prediction
End-to-End MLOps

Machine Learning Operations (AIMLCZG523) — Assignment 01
Experimental Learning: model development, CI/CD, containerisation, Kubernetes deployment and monitoring

Name	Ayush Chauhan
BITS ID	2025AE05684
Date	20 October 2026
Repository	`github.com/Phenomenal-19/MlOPS`

0.902	**87**	**5**	**2**
CV ROC-AUC Logistic Regression	automated tests 84% line coverage	CI jobs, all green on the latest run	API replicas on Kubernetes (kind)

Dataset: UCI Heart Disease (Cleveland) · Python 3.12 · scikit-learn · MLflow · FastAPI · Docker · Kubernetes · GitHub Actions · Prometheus · Grafana

Summary and assignment coverage

This project builds a classifier that predicts the presence of heart disease from 13 clinical measurements and operates it the way a production service would be operated: every step is scripted, every experiment is tracked, the model is packaged with its preprocessing, every push to GitHub is linted, tested, containerised and deployed to a throw-away Kubernetes cluster, and the running service exports metrics that feed a Grafana dashboard. Three models were tuned with cross-validation; **Logistic Regression** was selected (cross-validated ROC-AUC 0.902, held-out recall 0.93) and is served at `POST /predict`.

#	Assignment task	Marks	Delivered	Section
1	Data acquisition and EDA	5	Download + cleaning scripts, five EDA figures	3
2	Feature engineering and models	8	Shared pipeline; 3 models; grid search; 5-fold CV; 4 metrics	4
3	Experiment tracking	5	MLflow: parameters, metrics, plots, tuning table, model per run	5
4	Packaging and reproducibility	7	joblib + MLflow model, pinned requirements, clean runs on two machines	6
5	CI/CD and automated testing	8	87 pytest tests, ruff, GitHub Actions, artifacts, quality gate	7
6	Model containerisation	5	FastAPI + Dockerfile, /predict, smoke test	8
7	Production deployment	7	Kubernetes manifests, Ingress, verified rollout	9
8	Monitoring and logging	3	Prometheus, Grafana dashboard, JSON request logs	10
9	Documentation and reporting	2	This report, README, architecture diagram	1, 2

1. Architecture

Data and ML (lane 1). A download script fetches the Cleveland file; preprocessing produces a cleaned CSV; one scikit-learn pipeline (imputation, scaling, encoding) is shared by training and serving. Training tunes three models and logs every run to MLflow; packaging exports the best run to `models/`. **CI/CD (lane 2).** A GitHub Actions workflow lints, tests, builds and tests the Docker image, deploys to a kind cluster and trains and packages the model, failing on any error. **Serving (lane 3).** Requests enter an nginx Ingress, are load-balanced by a Service across two FastAPI pods. **Monitoring (lane 4).** Prometheus scrapes each pod's `/metrics`; Grafana renders a provisioned dashboard.

Figure 1 (next page) shows the whole system; the remaining sections follow its lanes.

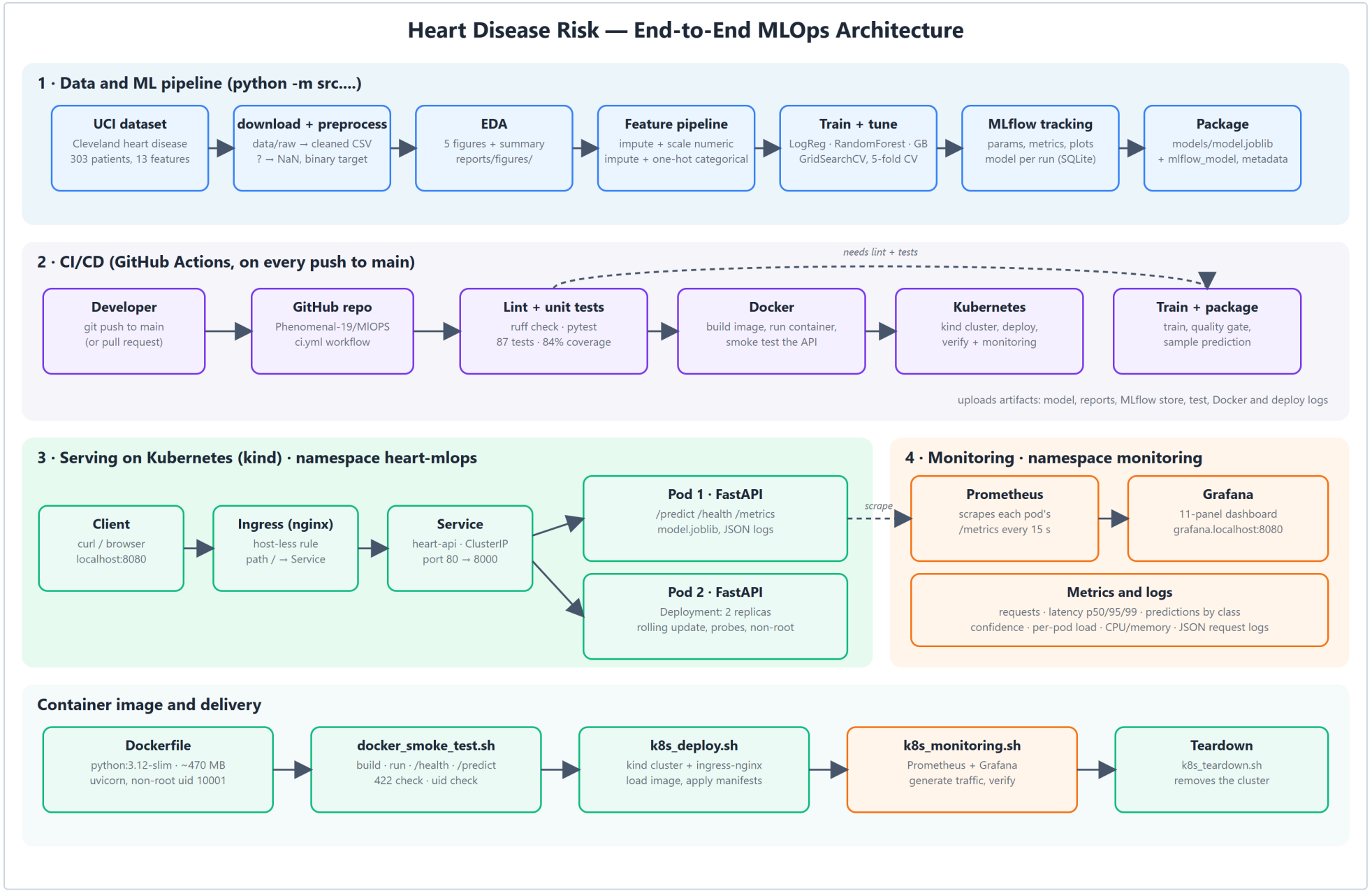


Figure 1 — End-to-end architecture: ML pipeline, CI/CD, Kubernetes serving and monitoring.

2. Setup and installation

Everything runs from a clean checkout using only the pinned requirement files. Prerequisites: Python 3.12, git, and Docker for the container and Kubernetes steps (used inside WSL here). `kind` and `kubectl` are installed automatically into `~/.local/bin` without administrator rights. On a plain Linux machine, `bash scripts/lab_run.sh` creates the environment and runs the whole ML pipeline, tests and a local API check in one command.

```
git clone https://github.com/Phenomenal-19/MlOPS.git && cd MlOPS
python -m venv .venv && .venv\Scripts\activate          # Linux/WSL: source .venv/bin/activate
pip install -r requirements.txt                          # pinned: pandas, scikit-learn, mlflow, fastapi ...

python -m src.data.download && python -m src.data.preprocess      # dataset -> data/processed/
python -m src.eda.run_eda                                         # figures -> reports/figures/
python -m src.models.train && python -m src.models.package        # tune, track (MLflow), export
python -m src.predict --input sample_input.json                   # local inference

bash scripts/docker_smoke_test.sh        # build image, run container, verify the API
bash scripts/k8s_deploy.sh               # kind cluster + ingress + 2 replicas + verification
bash scripts/k8s_monitoring.sh           # Prometheus + Grafana + traffic + verification
```

File	Purpose
requirements.txt / requirements-serve.txt / requirements-dev.txt	full environment / slim serving image / tests and lint
src/ (data, eda, features, models, api, predict)	all code, run as modules (`python -m src...`)
test/	pytest suite: data, features, training, API, metrics, manifests
k8s/, k8s/monitoring/	Deployment, Service, Ingress, kind config; Prometheus and Grafana
scripts/	Docker smoke test, Kubernetes deploy / monitoring / teardown, traffic generator

3. Data acquisition and exploratory analysis

The **UCI Heart Disease (Cleveland)** dataset has 303 patients, 13 clinical features and a 0–4 severity label. `src/data/download.py` downloads the raw file and adds the column header, so the data can be recreated at any time (the raw file is not committed; the cleaned CSV is). `src/data/preprocess.py` converts the dataset's `?` placeholders to missing values, turns severity into a binary `target` (0 = no disease, 1 = disease) and removes duplicates. Only six values are missing (`ca`: 4, `thal`: 2). They are deliberately left as missing in the cleaned file and imputed *inside* the model pipeline, so imputation statistics are learned from training folds only and cannot leak.



Figure 2 — Class balance: 45.9% disease (139 of 303); no resampling needed.

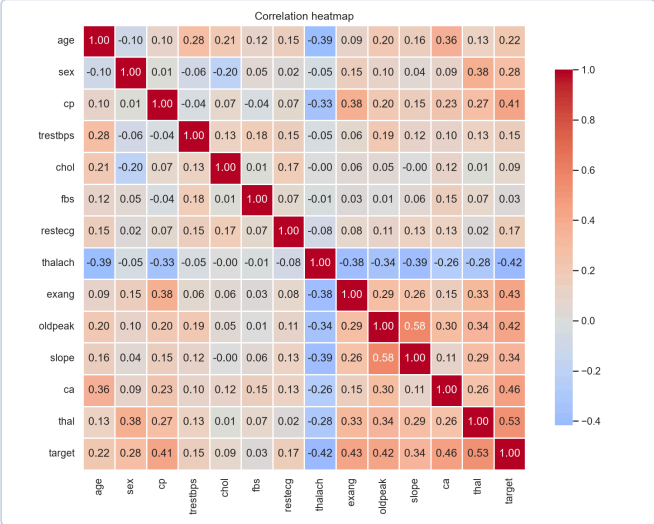


Figure 3 — Correlation heatmap including the target.

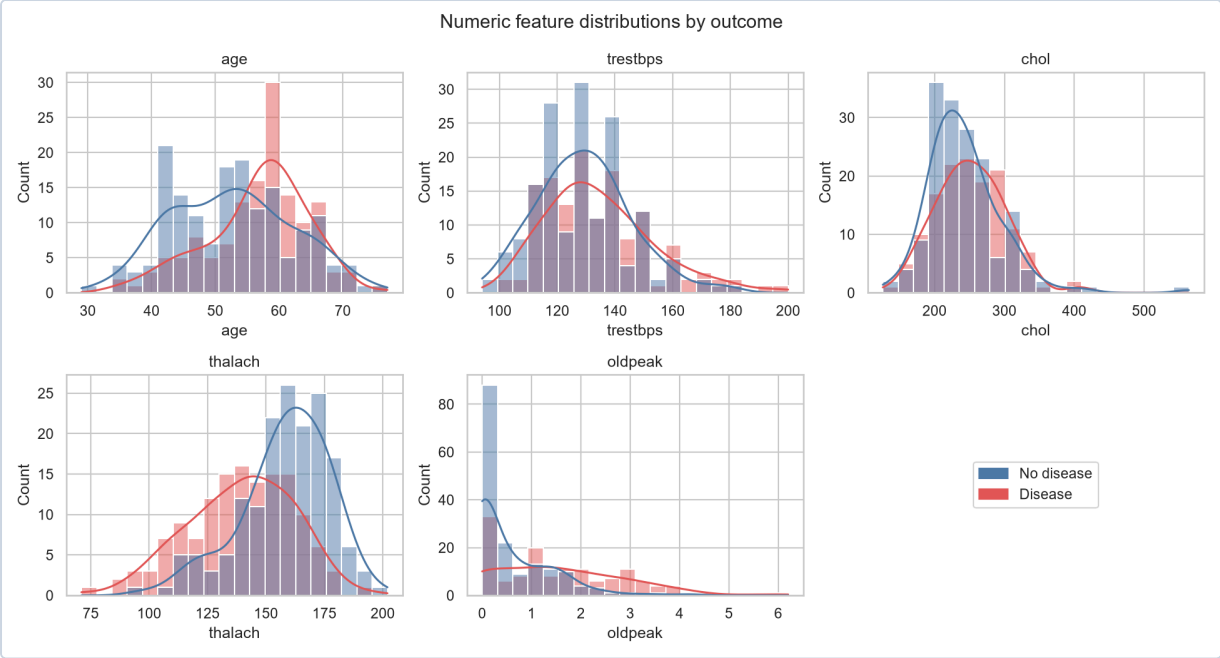


Figure 4 — Numeric features by outcome: patients with disease reach lower peak heart rate (thalach) and higher ST depression (oldpeak).

Findings. The most informative features are `thal` (r = 0.53), `ca` (0.46), `exang` (0.43), `oldpeak` (0.43) and `cp` (0.41); maximum heart rate `thalach` is negatively related (−0.42). Cholesterol (0.09) and fasting blood sugar (0.03) carry little signal on their own. The two classes are close to balanced, so accuracy is informative, but recall is tracked closely because missing a sick patient is the costly error.

4. Feature engineering and model development

A single `ColumnTransformer` (`src/features.py`) is the only preprocessing path for both training and serving: numeric columns (age, resting blood pressure, cholesterol, peak heart rate, ST depression) get median imputation and standard scaling; categorical columns (sex, chest pain, fasting sugar, resting ECG, exercise angina, slope, vessels, thalassemia) get most-frequent imputation and one-hot encoding with unseen categories ignored. It is part of the saved model, so inference reproduces training exactly.

Protocol. A stratified 80/20 split (242 train / 61 test, seed 42); the test set is never used for tuning. Each of three candidate models is tuned with `GridSearchCV` (stratified 5-fold, scored on ROC-AUC) on the training set; the tuned pipeline is then cross-validated for accuracy, precision, recall, F1 and ROC-AUC and finally scored once on the test set.

Model	Search grid	Selected
Logistic Regression	C ∈ {0.01, 0.1, 1, 10}	C = 1
Random Forest	trees {100, 300} × depth {3, 5, None} × min leaf {1, 3, 5}	300 trees, depth None, min leaf 5
Gradient Boosting	trees {50, 100, 200} × learning rate {0.03, 0.1} × depth {2, 3}	50 trees, rate 0.03, depth 2

Model	CV acc.	CV prec.	CV recall	CV F1	CV ROC-AUC	Test acc.	Test recall	Test ROC-AUC
Logistic Regression	0.847	0.877	0.783	0.825	0.902	0.885	0.929	0.966
Random Forest	0.81	0.81	0.774	0.787	0.902	0.852	0.893	0.948
Gradient Boosting	0.822	0.831	0.783	0.799	0.883	0.885	0.857	0.938

CV = mean of 5 stratified folds on the training set. Test = held-out 61 patients.

Model selection. Logistic Regression and Random Forest tie on cross-validated ROC-AUC (0.902), but Logistic Regression is more stable across folds (standard deviation 0.014 versus 0.033), has the best cross-validated precision and the best held-out recall (0.93), is the simplest to explain, and is cheap to serve. Gradient Boosting was weakest on ROC-AUC. Because the test set has only 61 patients, its numbers are noisy, so the cross-validated scores are treated as the primary evidence.

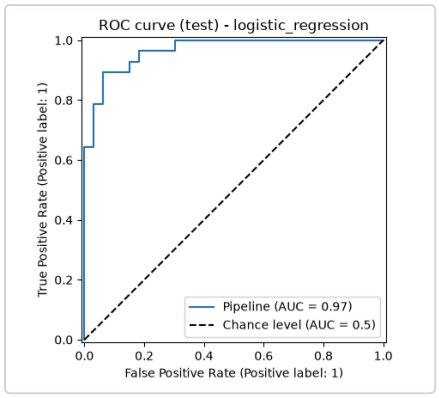


Figure 5 — ROC curve (test), AUC 0.97.

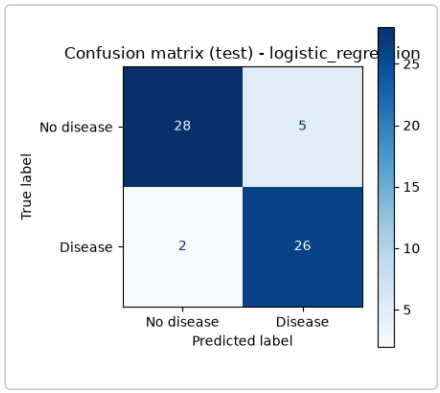


Figure 6 — Confusion matrix (test).

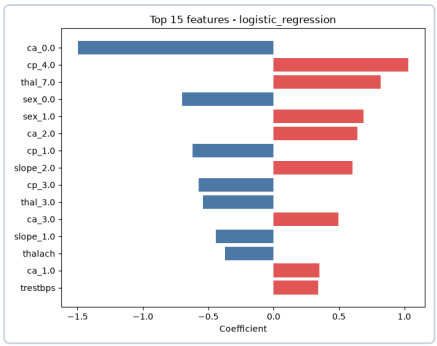


Figure 7 — Largest model coefficients.

5. Experiment tracking

Every candidate is logged to an MLflow experiment (`heart-disease-classification`, SQLite backend) as its own run: tuned parameters, seed, fold count, split sizes and class rate; cross-validated mean and standard deviation of five metrics plus the test metrics; the ROC curve, confusion matrix and coefficient/importance plots; the full grid-search table as `tuning/grid_search_results.csv`; and the fitted pipeline with an input example and signature. The best run by cross-validated ROC-AUC is tagged `best_model=true`, which is how the packaging step finds it.

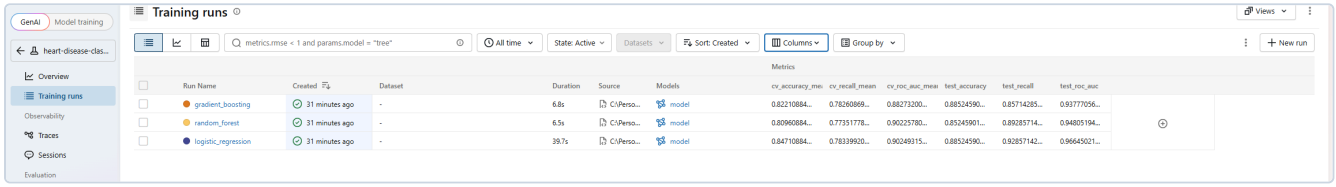


Figure 8 — MLflow runs table with CV and test metrics for the three models.

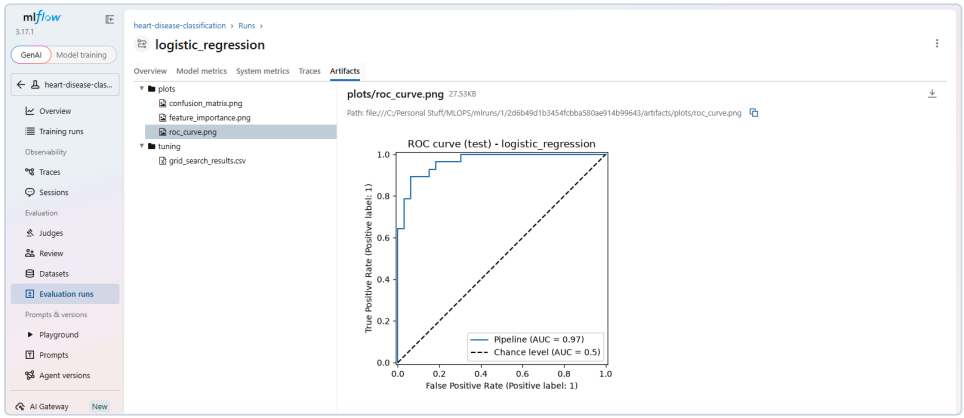


Figure 9 — Logged artifacts of the best run: plots, tuning table and the ROC curve.

6. Model packaging and reproducibility

`src/models/package.py` exports the tagged run to `models/model.joblib` (preprocessing and classifier in one object, used by the API without any MLflow dependency), `models/mlflow_model/` (MLflow format with signature and environment files) and `models/metadata.json` (run id, metrics, feature list, library versions). The export asserts that it reproduces the tracked model's predictions exactly, and it can be re-run over an existing export.

- **Pinned environments.** All dependencies are pinned; the serving image installs a separate slim file with matching versions.
- **Clean-room proof.** Source files only were copied to an empty folder, a new virtual environment was built from `requirements.txt`, and the whole pipeline (download, preprocess, EDA, train, package, predict) was run; it reproduced the same metrics and the same sample prediction.

- **Second environment.** The one-command runner `scripts/lab_run.sh` was then run on a separate Linux virtual-lab machine (Python 3.12.9). It created its own environment, installed the pinned requirements, downloaded the dataset, trained and tracked the models, packaged the best one, passed lint, all 87 tests and a live API check, and reproduced the same metrics and the same sample prediction (Figures 10 and 11; the training output is in `screenshots/04_lab_clean_run_2_training.png`).
- **Serialisation.** MLflow 3 defaults to the `skops` format, which rejected the fitted pipeline; `cloudpickle` is used instead. Pickle-based files can run code on load, so only trusted, self-built models should be loaded.

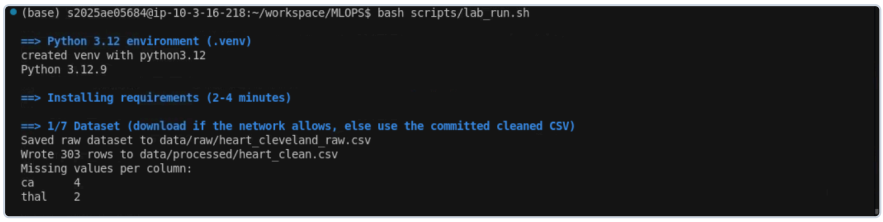


Figure 10 — One-command run on the second machine: environment created, requirements installed, dataset downloaded and cleaned.

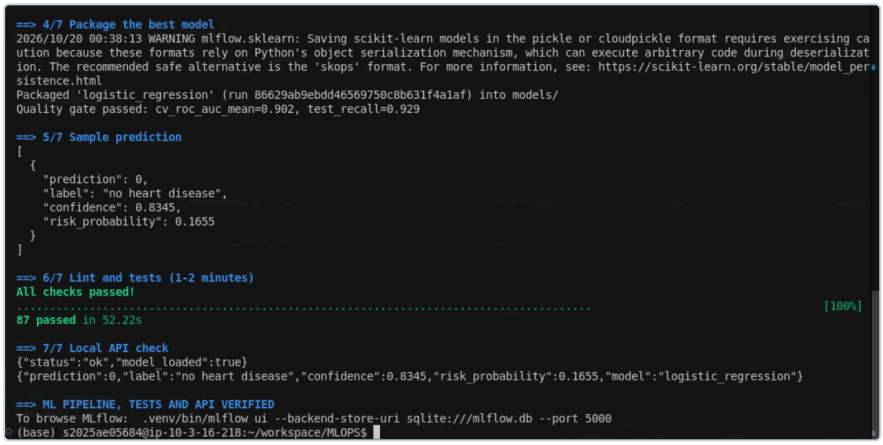


Figure 11 — Same run: model packaged and quality gate passed, lint, 87 tests and the API check all verified.

7. CI/CD pipeline and automated testing

Tests. 87 pytest tests (12 files, 84% line coverage) use small synthetic data, so they need no network: cleaning rules, preprocessing behaviour (no NaNs, standardised columns, unseen categories), training and metric ranges, prediction schema and confidence, an MLflow logging integration test, API validation and status codes, metrics and log output, and consistency checks across the Kubernetes manifests and Grafana dashboard. **Lint:** ruff.

Workflow (`.github/workflows/ci.yml`, on every push and pull request): `lint` and `test` run first; `docker` builds the image and runs the container smoke test; `kubernetes` creates a kind cluster, deploys, and verifies the API and monitoring; `train` retrains, packages, applies a **quality gate** (cross-validated ROC-AUC ≥ 0.85, test recall ≥ 0.75) and runs a sample prediction. Each run uploads seven artifacts (model, reports, MLflow store, test results, coverage, Docker and Kubernetes logs). Every step exits non-zero on failure, so the workflow stops: a deliberately broken test and a lint violation each made their checks fail, an impossible quality threshold made the gate fail with a clear message, and the first real CI run failed at the packaging step (Section 11).

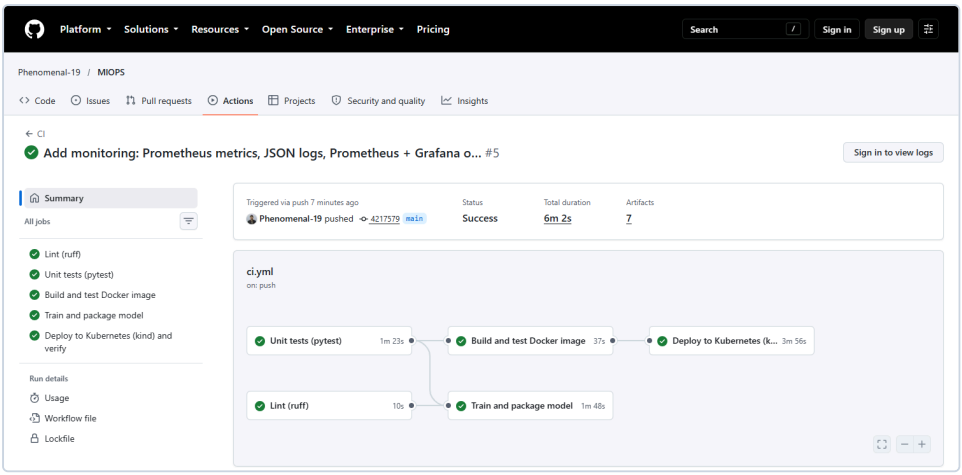


Figure 12 — Latest CI run: all five jobs green in 6 min 2 s.

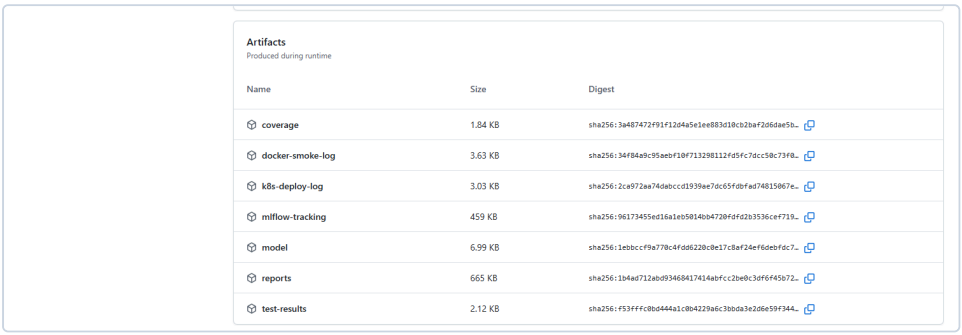


Figure 13 — Artifacts uploaded by that run (model, reports, MLflow store, logs).

8. Model containerisation

The model is served by a FastAPI application (`src/api/app.py`). `POST /predict` accepts one patient as JSON and returns the prediction, a label, the **confidence** (probability of the predicted class) and the risk probability. Inputs are validated against clinical ranges and allowed values; unknown fields and out-of-range values return HTTP 422; `ca` and `thal` may be omitted. The `Dockerfile` uses `python:3.12-slim`, installs only the serving requirements, copies only inference code and the packaged model, runs as a non-root user (uid 10001) and defines a health check; the image is about 470 MB. `scripts/docker_smoke_test.sh` builds it, runs it, and checks `/health`, a prediction, the 422 behaviour and the non-root user.

```
$ curl -X POST localhost:8000/predict -H "Content-Type: application/json" -d @sample_input.json
{"prediction":0,"label":"no heart
disease","confidence":0.8345,"risk_probability":0.1655,"model":"logistic_regression"}
```

```
==> Starting container on port 8000
==> Waiting for /health
{"status":"ok","model_loaded":true}
==> POST /predict with sample_input.json
{"prediction":0,"label":"no heart disease","confidence":0.8345,"risk_probability":0.1655,"model":"logistic_regression"}
response schema OK
==> Invalid input must be rejected with 422
got 422 as expected
==> Container runs as non-root
uid=10001
==> Container logs
INFO:     Started server process [1]
INFO:     Waiting for application startup.
2026-10-18 21:13:38,479 INFO heart-api Loaded model 'logistic_regression' from /app/models/model.joblib
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
2026-10-18 21:13:38,520 INFO heart-api req=3d85aeca GET /health status=200 latency_ms=1.7
INFO:     172.17.0.1:54364 - "GET /health HTTP/1.1" 200 OK
2026-10-18 21:13:38,527 INFO heart-api req=d1ab1a90 GET /health status=200 latency_ms=0.6
INFO:     172.17.0.1:54374 - "GET /health HTTP/1.1" 200 OK
2026-10-18 21:13:38,569 INFO heart-api prediction=0 confidence=0.835
INFO:     172.17.0.1:54382 - "POST /predict HTTP/1.1" 200 OK
2026-10-18 21:13:38,570 INFO heart-api req=a6980250 POST /predict status=200 latency_ms=33.8
INFO:     172.17.0.1:54388 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
2026-10-18 21:13:38,629 INFO heart-api req=93fd2287 POST /predict status=422 latency_ms=1.2
==> ALL DOCKER CHECKS PASSED
```

Figure 14 — Docker build/run proof: health, prediction, 422 rejection, non-root user, all checks passed.

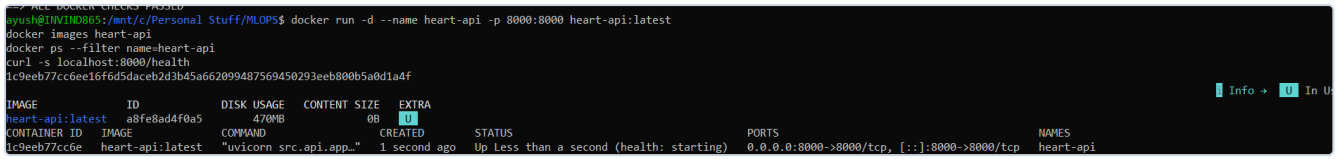


Figure 15 — Image size and the running container.

9. Production deployment on Kubernetes

The container is deployed to a local **kind** cluster (Kubernetes 1.32) with manifests in `k8s/`: a **Deployment** with two replicas (rolling updates with zero unavailable pods, readiness and liveness probes on `/health`, CPU and memory requests and limits, non-root, no privilege escalation), a ClusterIP **Service**, and an nginx **Ingress** exposed on `localhost:8080`. `scripts/k8s_deploy.sh` creates the cluster, loads the image, installs the ingress controller, applies the manifests, waits for the rollout and verifies `/health`, `/predict` and the 422 path through the Ingress; it also sends 20 requests to show traffic spread across both pods.

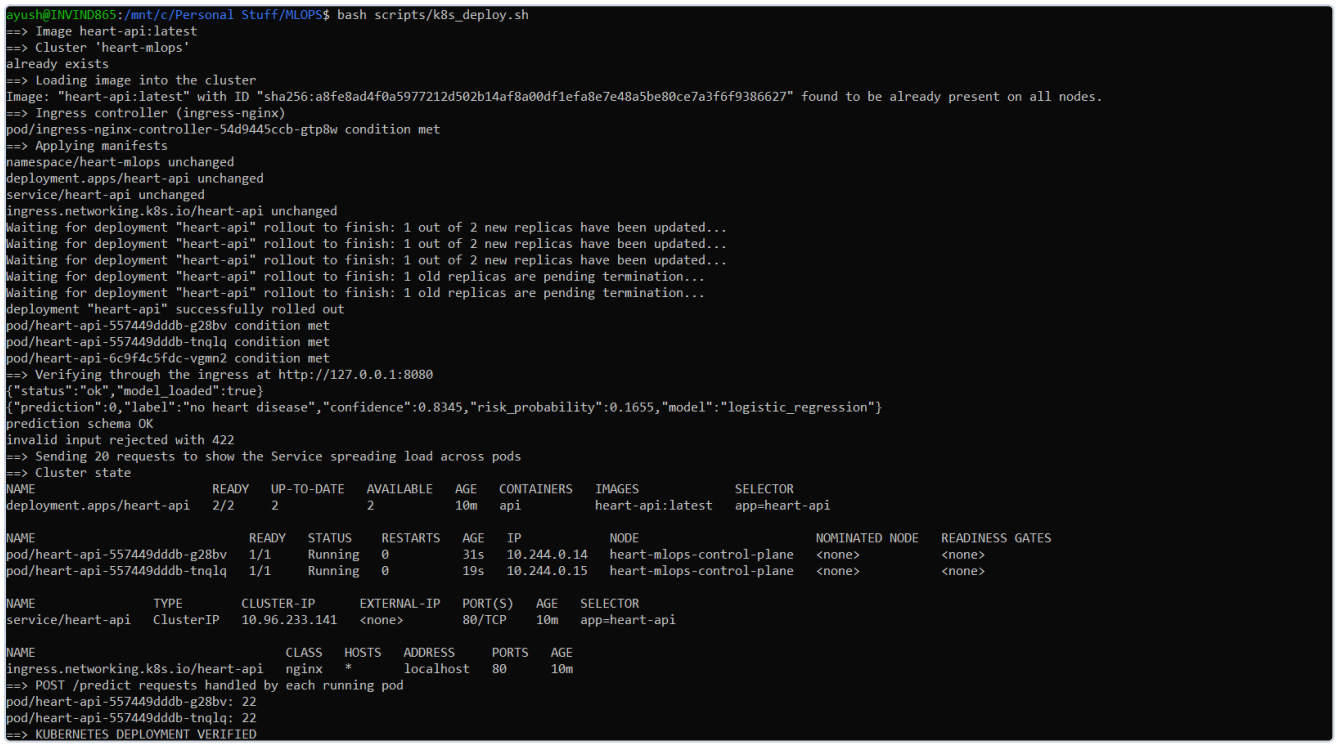


Figure 16 — Deployment output: 2/2 pods, 20 requests spread across the pods, verified.

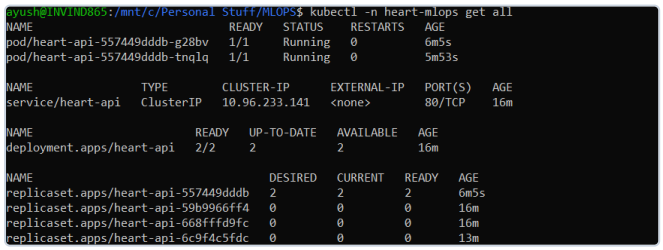


Figure 17 — kubectl get all.

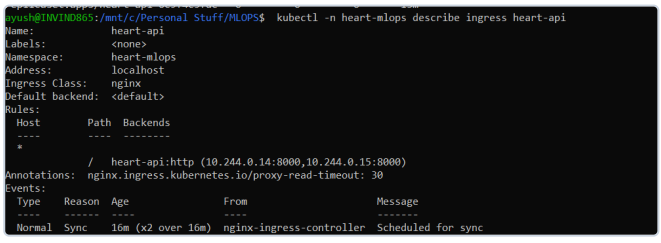


Figure 18 — Ingress routes / to heart-api:http.



Figure 19 — POST /predict through http://localhost:8080 (the ingress): request URL, code 200 and response.

10. Monitoring and logging

Metrics. The API exports Prometheus metrics at `/metrics`: request counts by method, route template and status; request-latency histograms; predictions by predicted class; a prediction-confidence histogram; and the loaded model. Route templates and small enumerations keep the number of time series bounded (all unknown URLs share one label). **Logs.** Every request is logged with id, method, path, status and latency; in the cluster the format is one JSON object per line, and the constant health and metrics probes are logged at debug level so real traffic stays visible.

Stack. Prometheus discovers each API pod through the Service endpoints (read-only RBAC limited to one namespace) and scrapes every 15 s. Grafana is provisioned from files with a Prometheus datasource and an 11-panel dashboard: totals, request rate, error rate, pods up, status codes, latency percentiles, predictions by class, average confidence, per-pod load, and CPU and memory. In the demonstration run the two pods shared traffic almost equally, about 10% of requests were deliberately invalid (shown as the error rate), and mean prediction latency was about 14 ms.



Figure 20 — Grafana dashboard with live traffic from the load generator.

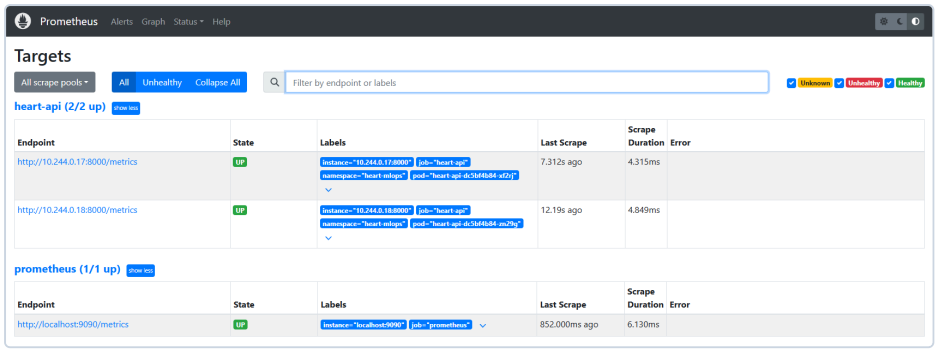


Figure 21 — Prometheus targets: both API pods are UP (scraped every 15 s).

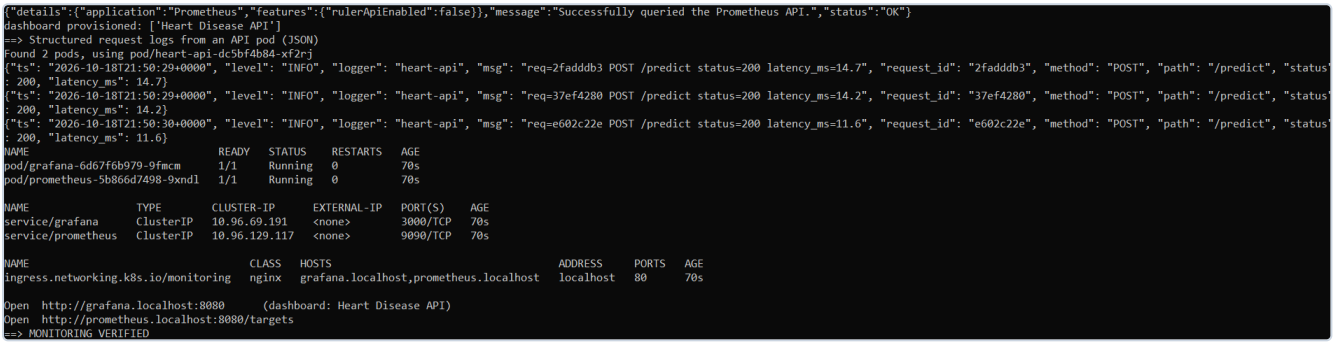


Figure 22 — Structured JSON request logs, and the monitoring pods, services and ingress.

11. Challenges and lessons learned

- **CI found a bug local runs missed.** The first CI run failed at packaging: MLflow refuses to save into a non-empty directory, and CI starts from a checkout that already contains `models/`. I reproduced it locally by re-running the step, made the export replace the old one, and added a test that exports twice.
- **Line endings.** Shell scripts written on Windows had CRLF endings that would break in WSL. They were converted and pinned with `.gitattributes`, and syntax-checked in the target shell before first use.
- **Stale image on redeploy.** The deploy script originally built the image only if it was missing, so code changes would not reach the cluster. It now always builds (cached layers keep it fast) and restarts the deployment on re-runs.
- **Verification must exercise the real path.** Docker, Kubernetes and monitoring checks run through the same entry points a user would (container port, Ingress, Prometheus API), and the manifests are additionally covered by consistency tests.

12. Conclusion, limitations and future work

The project delivers a complete, automated path from raw data to a monitored service: reproducible training with tracked experiments, a tested and containerised API, a CI pipeline that gates every change, a verified Kubernetes deployment and a live dashboard. **Limitations:** the dataset is small (303 patients) and the test set noisy; the model is a teaching example and not a clinical tool; the Kubernetes cluster is local, Grafana uses demo credentials, metric storage is ephemeral, and `/metrics` is exposed through the same Ingress. **Future work:** a model registry with staged promotion, drift and data-quality monitoring, alerting rules, autoscaling, decision-threshold tuning for higher recall, and deployment to a managed cloud cluster with TLS and authentication.

Item	Location
Code repository	`https://github.com/Phenomenal-19/MlOPS`
Local API (after `k8s_deploy.sh`)	`http://localhost:8080` (docs at `/docs`)
Dashboards	`http://grafana.localhost:8080` · `http://prometheus.localhost:8080/targets`
CI runs and artifacts	GitHub repository → Actions tab
Screenshots / report sources	`screenshots/`, `docs/` in the repository